Differenze tra i linguaggi

C · C++ · Java · Python

Appunti di Ingegneria del Software

Funzioni virtuali & late binding — Template & generics — Strategy pattern

Rielaborazione trasversale delle slide del corso

Anno 3

Contents

Chapter 1 Introduzione: perché confrontare i linguaggi — Page 3

1.1 I quattro linguaggi in una pagina 3

1.2 Modello di esecuzione: cosa succede al codice 4

Chapter 2 Binding e funzioni virtuali — Page 5

2.1 Early binding vs late binding 5
Il confronto fondamentale sul dispatch — 6

2.2 Funzioni virtuali e overriding 6
Overriding vs hiding vs overloading — 7

2.3 Sintassi nei quattro linguaggi 7
C++ — virtuale esplicito — 7 • Java — virtuale di default — 8 • C — niente OOP: puntatori a funzione — 8 • Python — tutto dinamico (duck typing) — 8

2.4 Come si implementa il late binding: la vtable 9

2.5 Virtual vs non-virtual: la questione dell'efficienza 9

2.6 Esempio: tracciare l'output (dalle slide) 10

Chapter 3 Template e generics — Page 11

3.1 Il problema: container e tipizzazione 11

3.2 Generics e template: la differenza profonda 11

3.3 Function template 12
Il classico tranello `char*` — 13

3.4 Class template 13
Istanziazione implicita ed esplicita — 14

3.5 Genericità nei quattro linguaggi 14
C — nessun generics: macro, `void*`, `_Generic` — 14 • C++ — template (già visti) — 14 • Java — generics con erasure — 14 • Python — duck typing e type hints — 15

Chapter 4 Strategy pattern e costrutti abilitanti — Page 16

4.1 L'idea dello Strategy pattern 16

4.2 I costrutti Java che lo rendono elegante 16
Classi annidate, interne, locali, anonime — 17 • Interfacce — 19 • Classi anonime e listener — 20

4.3 Lo Strategy nei quattro linguaggi 20
Java — interfaccia + (anonima/lambda) — 20 • C++ — classe astratta, `std::function` o template — 21 • C — puntatori a funzione (lo Strategy "originale") — 21 • Python — funzioni di prima classe (quasi banale) — 21

4.4 Ereditarietà e polimorfismo: quadro comparativo 22

Chapter 5 Tabelle riassuntive per l'esame Page 23
5.1 Modello di esecuzione e tipizzazione 23
5.2 Polimorfismo e dispatch (funzioni virtuali) 23
5.3 Genericità (template / generics) 23
5.4 OOP: ereditarietà e interfacce (strategy) 24
5.5 Le domande "trappola" tipiche del prof 24

Chapter 1

Introduzione: perché confrontare i linguaggi

Le slide del corso presentano tre argomenti (**funzioni virtuali e late binding**, **template**, **strategy pattern**) usando principalmente C++ e Java come linguaggi di riferimento. Il professore, però, chiede ripetutamente di ragionare sulle *differenze tra i linguaggi*: perché una certa feature esiste in C++ ma non in Java, come si emula un meccanismo in C che non ha l'orientamento agli oggetti, cosa cambia in un linguaggio a tipizzazione dinamica come Python.
Questi appunti riorganizzano quindi *tutti* gli argomenti delle slide mettendo sistematicamente a confronto **C, C++, Java e Python**. L'idea è che ogni meccanismo (l'overriding, il template, lo strategy) va capito non come un dettaglio sintattico di un linguaggio, ma come *una risposta a un problema di ingegneria del software* (riuso, estensibilità, disaccoppiamento) che ciascun linguaggio risolve con scelte diverse.

1.1 I quattro linguaggi in una pagina

Prima di entrare nei dettagli conviene fissare le coordinate fondamentali dei quattro linguaggi, perché quasi tutte le differenze che vedremo discendono da queste scelte di progetto.

Definition 1.1.1: Tipizzazione statica vs dinamica

Un linguaggio è a **tipizzazione statica** se il tipo di ogni espressione è noto e controllato a *tempo di compilazione* (C, C++, Java). È a **tipizzazione dinamica** se i tipi sono associati ai *valori* e controllati a *tempo di esecuzione* (Python).

Ortogonale a questa c'è la distinzione **forte/debole**: un tipaggio è *forte* se il linguaggio impedisce (o rende esplicite) le conversioni di tipo pericolose. C è statico ma relativamente *debole* (i cast e l'aritmetica dei puntatori permettono di reinterpretare la memoria); Java e Python sono *forti*.

Aspetto	**C**	**C++**	**Java**	**Python**
Paradigma	imperativo / procedurale	multiparadigma (proc. + OOP + generico)	OOP (class-based)	multiparadigma (OOP + funzionale)
Tipizzazione	statica, debole	statica, forte	statica, forte	dinamica, forte
Esecuzione	compilato → nativo	compilato → nativo	compilato → bytecode → JVM	interpretato → bytecode → VM (CPython)
Memoria	manuale (`malloc`/`free`)	manuale + RAII / smart pointer	garbage collector	GC (reference counting + cicli)
Dispatch metodi	assente (puntatori a funzione)	statico di default, dinamico se `virtual`	**sempre** dinamico	**sempre** dinamico (duck typing)
Genericità	macro, `void*`, `_Generic`	template (monomorfizzazione)	generics (type erasure)	duck typing + type hints
Ereditarietà	assente	multipla (con `virtual`)	singola + interfacce multiple	multipla (MRO C3)

Note:

Tieni a mente questa tabella: è la *chiave di lettura* di tutto il resto. Ogni volta che una feature "esiste in un linguaggio e non nell'altro" la spiegazione sta quasi sempre in una di queste righe (tipizzazione, modello di esecuzione, gestione della memoria).

1.2 Modello di esecuzione: cosa succede al codice

- **C e C++** sono *compilati a codice macchina nativo*. Il compilatore traduce direttamente in istruzioni della CPU; il binario gira "sul metallo". Questo dà massime prestazioni e controllo, ma nessuna rete di sicurezza a runtime.
- **Java** è compilato in un formato intermedio (il *bytecode*) eseguito da una macchina virtuale (JVM). La JVM aggiunge controlli a runtime (array bounds, type safety, garbage collection) e porta il compromesso "write once, run anywhere".
- **Python** (CPython) compila il sorgente in bytecode che poi *interpreta* in un ciclo di valutazione. Nessun controllo statico dei tipi: gli errori di tipo emergono solo quando la riga viene eseguita.

Questa gerarchia (nativo → bytecode+VM → interpretato) spiega il trade-off ricorrente del corso: **più controllo e prestazioni** da un lato (C/C++), **più sicurezza e produttività** dall'altro (Java/Python).

Chapter 2

Binding e funzioni virtuali

Questo capitolo copre la lezione *Virtual Functions and Late Bindings*. Il concetto centrale è il **binding**: il momento in cui una chiamata di funzione viene associata al codice che effettivamente verrà eseguito.

2.1 Early binding vs late binding

Definition 2.1.1: Binding

Il **binding** è l'associazione tra una *chiamata* a funzione (il punto nel codice dove appare l'invocazione) e la *definizione* della funzione (l'indirizzo di memoria del codice da eseguire).

Definition 2.1.2: Early binding (compile-time binding)

L'associazione chiamata→codice avviene a **tempo di compilazione**. Il compilatore, incontrando la chiamata, scrive un'istruzione macchina di `call` verso un indirizzo ben preciso e fisso. È il comportamento standard delle funzioni in C e delle funzioni *non* virtuali in C++.

Definition 2.1.3: Late binding (run-time binding)

L'associazione chiamata→codice viene rimandata al **tempo di esecuzione**: il compilatore non genera una `call` a un indirizzo fisso, ma codice che *calcola* a runtime quale funzione invocare, in base al tipo *effettivo* (dinamico) dell'oggetto. È il meccanismo che rende possibili le funzioni virtuali.

Note:

Uno dei principi centrali dell'OOP è determinare il comportamento degli oggetti *il più tardi possibile*, idealmente a runtime. Nei linguaggi imperativi non-OO (C, Pascal, ...) l'unico modo per farlo sono `if` o `switch` espliciti sul "tipo": le funzioni virtuali sono la soluzione elegante e automatica a questo problema.

2.1.1 Il confronto fondamentale sul dispatch

Linguaggio	Comportamento di default del dispatch
C	Nessun dispatch dinamico nativo. Le chiamate sono *sempre* in early binding. Il polimorfismo si emula a mano con **puntatori a funzione** (spesso raccolti in una `struct`, che diventa una vtable manuale).
C++	Early binding di **default**. Il late binding si attiva solo dichiarando il metodo `virtual`. Filosofia: "non paghi per ciò che non usi" (*zero-overhead*).
Java	Late binding **sempre**: tutti i metodi non statici sono virtuali per default. Si può impedire l'override con `final`.
Python	Late binding **sempre**, per natura del linguaggio dinamico: il metodo da chiamare è cercato a runtime nel dizionario della classe (duck typing). Non esiste il concetto di "virtuale/non virtuale".

2.2 Funzioni virtuali e overriding

Definition 2.2.1: Funzione virtuale

Un metodo *non statico* di una classe base che può essere **ridefinito** (*overridden*) in una classe derivata. La ridefinizione fa sì che, chiamando il metodo su un riferimento/puntatore alla classe base, venga eseguita la versione della classe *reale* dell'oggetto.

Definition 2.2.2: Overriding

Il processo con cui un metodo della classe base viene *sostituito* da un altro nella classe derivata. Avviene solo se:

- il metodo sovrascritto è virtuale;
- il metodo nella derivata ha **stessa signature** (e stesso tipo di ritorno) del metodo nella base.

Definition 2.2.3: Signature e prototipo

La **signature** di una funzione comprende: *nome*, *numero* dei parametri formali, *tipi* dei parametri formali. Il *tipo di ritorno NON fa parte della signature.*
La signature *insieme* al tipo di ritorno è detta **prototipo** della funzione.

Proposition 2.2.1 Le tre possibilità dell'invocazione virtuale

Quando una funzione virtuale f è invocata su un oggetto o di classe D derivata da B, ci sono tre casi:

1. f definita solo in D: viene invocata $D.f$;
2. f definita solo in B: viene invocata $B.f$ (ereditata da D);
3. f definita (overridden) sia in B sia in D: viene invocata $D.f$.

In tutti i casi conta la **classe reale** dell'oggetto, non la classe del riferimento che lo punta.

Note:

Quando si invoca una funzione virtuale su un oggetto il cui tipo è *noto a compile-time* (es. un oggetto locale, non un riferimento/puntatore), il comportamento è indistinguibile da quello di una funzione non virtuale: il vantaggio del polimorfismo emerge *solo* manipolando gli oggetti tramite riferimenti (Java) o puntatori (C++) alla classe base.

2.2.1 Overriding vs hiding vs overloading

Un punto su cui il prof insiste e che distingue i linguaggi:

- **Overriding**: stessa signature + metodo virtuale ⇒ sostituzione a runtime (polimorfismo).
- **Overloading**: stesso nome, signature *diversa* ⇒ funzioni distinte scelte a compile-time in base agli argomenti.
- **Hiding**: in C++, se nella derivata dichiaro un metodo con lo stesso nome ma signature diversa di uno della base, *tutti* i metodi con quel nome nella base diventano **invisibili** nella derivata (non c'è overloading tra base e derivata).

Situazione	**C++**	**Java**
Metodo non dichiarato virtuale, ridefinito nella derivata	Non è override: è **hiding** (early binding sul tipo statico).	Non esiste il caso: *tutti* i metodi sono virtuali.
Stesso nome, signature diversa nella derivata	I metodi omonimi della base diventano **invisibili** (nascosti).	Semplicemente **non** avviene override; i metodi restano visibili (overloading).
Tipo di ritorno diverso, stessa signature	Errore di compilazione (C++ ammette solo piccole variazioni: *covariant return types*).	Errore, salvo tipo di ritorno covariante.
Impedire l'override	Non c'è `final` sui metodi (storicamente); i metodi non-`virtual` semplicemente non si sovrascrivono.	Parola chiave `final` (sul metodo o sull'intera classe).

2.3 Sintassi nei quattro linguaggi

2.3.1 C++ — virtuale esplicito

In C++ un metodo è virtuale **solo se dichiarato tale**. I metodi non virtuali usano early binding; se provi a "sovrascriverli" nella derivata li stai solo *nascondendo*.

```
class B {
public:
    virtual void f() { std::cout << "B::f\n"; }  // virtuale
    void g()         { std::cout << "B::g\n"; }  // NON virtuale
};

class D : public B {
public:
    void f() override { std::cout << "D::f\n"; } // override (virtuale)
    void g()          { std::cout << "D::g\n"; } // HIDING, non override
};

int main() {
    B* p = new D();
    p->f();   // D::f  -> late binding (virtuale)
    p->g();   // B::g  -> early binding (tipo statico B)
    delete p;
}
```

Note:

In C++ (dal C++11) conviene sempre usare `override`: non cambia la semantica ma fa segnalare al compilatore se per errore *non* stai davvero sovrascrivendo (es. signature sbagliata). `final` su un metodo o classe impedisce ulteriori override/derivazioni (introdotto in C++11, quindi *oggi* un `final` esiste anche in C++, a differenza di quanto dicono le slide più vecchie).

2.3.2 Java — virtuale di default

In Java **ogni** metodo non statico è virtuale. Non serve alcuna parola chiave; l'override è sempre possibile (salvo `final`).

```
class B {
    public void f() { System.out.println("B::f"); }
}
class D extends B {
    @Override
    public void f() { System.out.println("D::f"); }
}
public class Main {
    public static void main(String[] args) {
        B p = new D();
        p.f();   // D::f  -> sempre late binding
    }
}
```

2.3.3 C — niente OOP: puntatori a funzione

In C non esistono classi né metodi virtuali. Il polimorfismo si *emula* mettendo puntatori a funzione dentro una `struct`: di fatto si costruisce a mano quella *tabella delle funzioni virtuali* che C++ e Java generano automaticamente.

```
/* "vtable" costruita a mano */
typedef struct Animal {
    void (*voice)(struct Animal* self);   // puntatore a funzione = metodo virtuale
} Animal;

void dog_voice(Animal* self) { printf("bau\n"); }
void cat_voice(Animal* self) { printf("miao\n"); }

int main() {
    Animal a = { dog_voice };   // "istanza" con il suo comportamento
    Animal b = { cat_voice };
    a.voice(&a);   // bau   -> dispatch scelto dal puntatore a runtime
    b.voice(&b);   // miao
}
```

Note:

Questo è *esattamente* ciò che il compilatore C++ fa sotto il cofano con la vtable (vedi sotto): la differenza è che in C++/Java è automatico e type-safe, in C è manuale ed è compito del programmatore mantenerlo corretto. Il prof cita proprio C e Pascal come linguaggi in cui il dispatch dinamico non esiste nativamente.

2.3.4 Python — tutto dinamico (duck typing)

In Python il dispatch è *sempre* dinamico e non serve dichiarare nulla: il metodo viene cercato a runtime seguendo la *Method Resolution Order* della classe. Non conta la "classe del riferimento" (che non è nemmeno tipizzato).

```
class Animal:
    def voice(self): print("???")

class Dog(Animal):
    def voice(self): print("bau")

class Cat(Animal):
    def voice(self): print("miao")

for x in (Dog(), Cat()):
    x.voice()      # bau, miao  -> sempre late binding
```

Definition 2.3.1: Duck typing

"If it walks like a duck and quacks like a duck, it's a duck". In Python non conta il *tipo* dichiarato di un oggetto ma solo il fatto che *possieda* il metodo/attributo richiesto. Il polimorfismo è quindi ancora più ampio di quello per ereditarietà: due classi *non* imparentate sono interscambiabili se offrono gli stessi metodi.

2.4 Come si implementa il late binding: la vtable

Definition 2.4.1: Virtual Function Table (VFT/vtable)

Ogni classe che ha funzioni virtuali possiede una tabella (VFT) contenente i **puntatori al codice** di ciascuna sua funzione virtuale (VFP1, VFP2, ...).

Definition 2.4.2: Virtual Function Table Pointer (VFTP/vptr)

Ogni *oggetto* di una classe con funzioni virtuali contiene un puntatore nascosto (invisibile all'utente) alla vtable della sua classe.

Proposition 2.4.1 Passi del late binding a runtime

Per eseguire la chiamata `p->VF2()` attraverso un puntatore `p` di tipo base:

1. dal puntatore si raggiunge l'oggetto;
2. nell'oggetto si legge il VFTP (vptr);
3. tramite il VFTP si raggiunge la vtable della classe *reale*;
4. si cerca la voce giusta (per `VF2`, l'entry VFP2);
5. si segue il puntatore trovato fino al codice e lo si esegue.

Nota: in questo contesto "puntatore" significa puntatore C++ *oppure* riferimento Java.

Note:

Java e Python usano meccanismi concettualmente identici: la JVM ha una tabella dei metodi per classe; CPython cerca il metodo nel `__dict__` della classe risalendo la MRO. Il costo dell'indirezione è lo stesso di cui parlano le slide.

2.5 Virtual vs non-virtual: la questione dell'efficienza

Se il polimorfismo è così comodo, perché non rendere tutti i metodi virtuali? È la domanda con cui si spiega la differenza filosofica **Java vs C++**:

- **Java** sceglie la semplicità: tutti i metodi virtuali, sempre.
- **C++** lascia la scelta all'utente (virtual/non-virtual). Il motivo è l'**efficienza**: il late binding costa (un'indirezione in più via vtable + impossibilità di fare *inlining*).

Proposition 2.5.1 Quando l'overhead conta

Il costo del late binding è di solito trascurabile. Diventa rilevante solo se un metodo *semplice* viene chiamato *moltissime* volte: in quei casi l'indirezione impedisce ottimizzazioni (inline) e può diventare un collo di bottiglia. Ambiti tipici: sistemi real-time, routine matematiche di base, programmazione di sistema di basso livello.

	C++	Java	Python
Default	non virtuale	virtuale	virtuale
Costo di default	zero-overhead	indirezione vtable	lookup in dizionario
Ottimizzabile via inline	sì (non-virtual)	sì (JIT devirtualizza)	no (interprete)
Controllo al programmatore	massimo	minimo	nullo

2.6 Esempio: tracciare l'output (dalle slide)

Le slide propongono un esercizio classico d'esame: dato un insieme di classi con metodi virtuali e non, capire quali chiamate compilano e quale output producono. Il ragionamento chiave:

Proposition 2.6.1 Regola per l'esercizio

- Una chiamata `p->m()` attraverso un puntatore a `A` **compila** solo se `A` (o un suo antenato) dichiara un metodo con la signature di `m`: il compilatore guarda il **tipo statico** del puntatore per decidere se la chiamata è legale.
- Se il metodo è `virtual`, *quale* versione venga eseguita dipende dal **tipo dinamico** (reale) dell'oggetto.
- Se il metodo *non* è virtuale, si esegue la versione del **tipo statico**.

Example 2.6.1 (Righe che non compilano)

Nell'esempio delle slide (classi `A..G`):

- `aG.f()` non compila se `G` non definisce un `f()` senza argomenti e `G` ha introdotto un `f(int)` che *nasconde* gli altri `f`.
- `a->f(3)` con `a` di tipo `A*` non compila: il compilatore cerca `A::f(int)`, che non esiste, anche se `a` punta davvero a un oggetto derivato che ce l'ha (conta il tipo statico).

Sono errori dovuti al fatto che la *legalità* della chiamata si decide sul tipo statico, mentre solo il *dispatch* è dinamico.

Chapter 3

Template e generics

Questo capitolo copre la lezione *Templates*. Il problema di partenza è il **riuso del codice**: molti programmi usano strutture dati generiche (stack, code, liste). Duplicare il codice per ogni tipo di elemento (una coda di clienti, una di messaggi, una di ordini) è insostenibile: ogni modifica va replicata ovunque. La soluzione è **parametrizzare il codice rispetto al tipo**.

3.1 Il problema: container e tipizzazione

Definition 3.1.1: Genericità (type parameterization)

Il meccanismo con cui una classe o una funzione viene *parametrizzata rispetto a un tipo*, così da definire *una sola volta* un algoritmo o una struttura dati che funziona per molti tipi diversi. Le due incarnazioni principali sono i **generics** (Java) e i **template** (C++).

Le slide legano la genericità al modello di tipizzazione:

- Nei linguaggi a **tipizzazione dinamica** (Python) i container generici, sia *omogenei* sia *eterogenei*, si costruiscono banalmente: una lista può contenere qualunque cosa, non c'è un tipo da fissare.
- Nei linguaggi a **tipizzazione statica** (C, C++, Java) un container "cablato" a un tipo limita il riuso. Le soluzioni pre-generics erano:
  - in **C**: `union`/variant record per l'eterogeneità (ma con un numero finito di alternative e cast "sporchi" in estrazione), oppure `void*` (che perde ogni type-safety);
  - difficoltà a *nascondere* l'implementazione: per iterare sul container serviva una variabile che rivelava la struttura interna.

Note:

Ecco perché i template "nascono statici": servono proprio a recuperare, nei linguaggi a tipizzazione statica, quella flessibilità che i linguaggi dinamici hanno gratis — ma *senza* rinunciare al controllo dei tipi a compile-time.

3.2 Generics e template: la differenza profonda

Proposition 3.2.1 Template = generazione di codice a compile-time

Un template è un *modello* secondo cui viene generato codice reale.

- Quando il compilatore incontra la *definizione* del template, **non genera codice**: fa i controlli possibili e rimanda il resto.
- Al punto di **istanziazione** (dove il template è usato con tipi concreti) il compilatore genera il codice *specifico* per quei tipi. Il corpo eseguito non è generico: è del tutto specializzato.

- Il codice generato per una data combinazione di parametri è prodotto **una sola volta** e riusato.

Questa è la distinzione capitale rispetto ai generics di Java:

	C++ template	**Java generics**
Quando agiscono	compile-time: **monomorfizzazione** (una copia di codice per ogni tipo)	compile-time: **type erasure** (una sola copia, i parametri di tipo spariscono)
Codice prodotto	`vector<int>` e `vector<double>` sono classi *diverse* e non imparentate	`List<Integer>` e `List<String>` condividono lo *stesso* bytecode (`List`)
Tipi primitivi	funzionano direttamente	**vietati**: solo tipi riferimento (serve `Integer`, non `int`)
Type checking	incompleto sulla definizione, completo all'istanziazione	completo sulla definizione, poi cancellato
A runtime	tipo pienamente noto	tipo **cancellato** (`new T[]` impossibile, no `instanceof List<String>`)
Dimensione binario	può crescere (code bloat)	costante
Prestazioni	massime (nessun boxing, inline)	boxing/unboxing per i primitivi

Definition 3.2.1: Monomorfizzazione

Strategia (C++, Rust) in cui il compilatore genera una *copia separata* del codice per ciascuna combinazione di parametri di tipo usata. Massime prestazioni, ma possibile aumento della dimensione del binario (*code bloat*).

Definition 3.2.2: Type erasure

Strategia (Java) in cui i parametri di tipo servono solo al type-checker e vengono *cancellati* dopo la compilazione: a runtime `List<String>` è semplicemente `List`. Vantaggio: un solo bytecode, compatibilità all'indietro. Svantaggio: informazioni di tipo non disponibili a runtime.

Note:

Le slide dicono che "i template non sono generics completi": non c'è typechecking completo sulla *definizione*, perché finché i tipi sono ignoti il compilatore non può verificare che, per esempio, `r = 2 * t` sia lecito. In Java invece un generic è controllato *subito* sulla definizione (grazie ai *bounds*, es. `<T extends Comparable<T>>`).

3.3 Function template

Definition 3.3.1: Function template

Definisce una *funzione generica* applicabile ad argomenti di tipi specificabili *in seguito*, alla chiamata. Il controllo di consistenza dei tipi è rimandato alla prima chiamata (rilassamento del type checking).

```
// dichiarazione: template + lista parametri tra <>
template <typename T1, typename T2>
void f(T1 t, T2& r) {
    r = 2 * t;   // legale solo per certi T1, T2: verificato all'istanziazione
}

template <class T>            // 'class' e 'typename' qui equivalenti
T maximum(T a, T b) {
    return a > b ? a : b;     // richiede che operator> sia definito su T
}

int main() {
```

```
    int x = 1, y = 2;
    f(y, x);                    // OK: deduce T1=int, T2=int
    // f(anA, x);               // ERRORE: A non ha operator* con 2
    std::cout << maximum(10, 15);   // deduce T=int
}
```

Proposition 3.3.1 Deduzione dei parametri

Nelle *function* template il compilatore di solito **deduce** da solo i tipi guardando gli argomenti della chiamata. Se non ci riesce (o la chiamata è ambigua) bisogna specificarli a mano: `maximum<double>(a, b)`.

3.3.1 Il classico tranello `char*`

Example 3.3.1 (max su stringhe C)

Con `maximum("Aladdin", "Jasmine")` il compilatore genera `maximum(char*, char*)`, che confronta gli *indirizzi* dei due puntatori, non i contenuti! Risultato inatteso. Si risolve con una **specializzazione**:

```
template <> char* maximum(char* a, char* b) {
    return strcmp(a, b) > 0 ? a : b;   // confronto vero delle stringhe
}
```

In C++ moderno non si presenta con `std::string` (che sovraccarica `operator>`). È un ottimo esempio del prof su "il template genera codice specifico che magari non fa ciò che credi".

3.4 Class template

Definition 3.4.1: Class template

Una classe parametrizzata rispetto a uno o più tipi. Sintassi analoga a quella dei function template.

Proposition 3.4.1 Istanziazione delle class template

Differenza rispetto alle function template: per le *classi* il compilatore **non deduce** i parametri (non ci sono argomenti da cui dedurli). Vanno *sempre* specificati esplicitamente: `A<int>`, `A<double>`.

```
template <typename T>
class A {
    T x, y;
public:
    A() : x(0), y(0) {}
    T getSum() { return x + y; }
    void setXY(T a, T b) { x = a; y = b; }
};

int main() {
    A<int>    iA;    // genera la classe A<int>
    A<double> dA;    // genera la classe A<double>  (tipo DIVERSO da A<int>)
}
```

Proposition 3.4.2 Specializzazioni = classi indipendenti

Ogni istanziazione con parametri diversi crea una nuova **specializzazione**. Due specializzazioni dello stesso template sono classi regolari e **completamente scorrelate**: `A<int>`, `A<double>`, `A<B>` non hanno alcuna relazione. Non puoi assegnare un `A<int>` a un `A<double>` senza una conversione esplicita.

3.4.1 Istanziazione implicita ed esplicita

```
Z<int> zi;                // istanziazione IMPLICITA (all'uso)
template class Z<int>;    // istanziazione ESPLICITA (forzata dal programmatore)
```

Note:

Sottigliezza dalle slide: dichiarare solo un *puntatore* `Z<int>* p;` **non** istanzia la classe (come dichiarare un puntatore a una struct incompleta). L'istanziazione scatta quando serve la definizione completa (es. creando un oggetto). Per questo, per i template, dichiarazione e definizione dei membri stanno di solito *nello stesso header*: al punto di istanziazione il compilatore deve avere sotto gli occhi tutto il codice.

3.5 Genericità nei quattro linguaggi

3.5.1 C — nessun generics: macro, `void*`, `_Generic`

C non ha genericità nel type system. Tre approcci storici:

```
/* 1) Macro: sostituzione testuale del preprocessore (nessun type-check) */
#define MAX(a, b) ((a) > (b) ? (a) : (b))

/* 2) void*: container generico ma type-unsafe, richiede cast a mano */
typedef struct { void** data; int n; } List;

/* 3) qsort della libreria standard: generico via void* + comparatore */
int cmp_int(const void* a, const void* b) {
    return (*(const int*)a) - (*(const int*)b);
}
qsort(arr, n, sizeof(int), cmp_int);

/* 4) _Generic (C11): dispatch statico sul tipo, embrione di overloading */
#define abs_g(x) _Generic((x), int: abs, double: fabs)(x)
```

Note:

Questi meccanismi corrispondono alle "soluzioni pre-OO per i container" citate nelle slide: `void*` e `union` danno genericità ed eterogeneità, ma a costo di perdere il controllo dei tipi e di dover inserire "messy test and cast code" all'estrazione dei valori.

3.5.2 C++ — template (già visti)

Monomorfizzazione a compile-time, massime prestazioni, base della *Standard Template Library* (STL: `vector`, `map`, `sort`, ...).

3.5.3 Java — generics con erasure

```
class Cell<A> {                 // A = parametro di tipo
    private A value;
    Cell(A value) { this.value = value; }
    A get() { return value; }
}
// uso senza cast:
Cell<String> c = new Cell<>("First");
String s = c.get();             // nessun cast necessario

// bounded type: vincolo sui parametri (impossibile nei template classici)
static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
```

Note:

Conseguenze dell'erasure da ricordare: non puoi fare `new T()`, `new T[n]`, né `o instanceof List<String>`; i generic *non* accettano primitivi (`List<int>` è illegale, serve `List<Integer>` con boxing).

3.5.4 Python — duck typing e type hints

Python non ha bisogno di generics a runtime: una funzione lavora su qualunque oggetto che supporti le operazioni usate.

```python
def maximum(a, b):
    return a if a > b else b       # funziona per int, float, str, ...

# I "generics" esistono solo come ANNOTAZIONI statiche (typing),
# controllate da tool esterni (mypy), MAI a runtime:
from typing import TypeVar, Generic
T = TypeVar("T")
class Cell(Generic[T]):
    def __init__(self, value: T) -> None:
        self.value = value
    def get(self) -> T:
        return self.value
```

Proposition 3.5.1 Confronto sintetico sulla genericità

- **C**: genericità simulata (macro/`void*`), niente type-safety.
- **C++**: template → codice specializzato a compile-time, veloce ma può gonfiare il binario.
- **Java**: generics → un solo bytecode (erasure), type-safe a compile-time ma info perse a runtime.
- **Python**: non servono (duck typing); i type hint sono solo documentazione controllabile offline.

Chapter 4

Strategy pattern e costrutti abilitanti

Questo capitolo copre la lezione *Focus on the strategy pattern in Java*. Lo scopo del prof è mostrare come un **design pattern** si realizzi concretamente nel codice, e come i costrutti del linguaggio (interfacce, classi interne/anonime, polimorfismo, funzioni di prima classe) ne siano l'infrastruttura.

4.1 L'idea dello Strategy pattern

Definition 4.1.1: Strategy pattern

Pattern comportamentale che **incapsula una famiglia di algoritmi** intercambiabili dietro una stessa interfaccia, permettendo di scegliere (anche a runtime) quale algoritmo usare, *senza* modificare il codice cliente che lo invoca. La "strategia" è un oggetto (o una funzione) passato a chi la usa.

Proposition 4.1.1 Cosa abilita lo Strategy

Lo Strategy si regge su uno di questi due meccanismi:

1. **polimorfismo per overriding + late binding**: la strategia è un oggetto che implementa un'interfaccia comune (Java, C++);
2. **funzioni di prima classe**: la strategia è direttamente una funzione passata come argomento (C con puntatori a funzione, Python con funzioni/lambda, C++ con `std::function`).

Il prof sottolinea che in Java lo Strategy è "polimorfismo via overriding e late binding", proprio *perché* Java non ha puntatori a funzione.

4.2 I costrutti Java che lo rendono elegante

Le slide introducono una serie di costrutti Java (classi annidate, interne, locali, anonime; interfacce con default/static) che servono a scrivere lo Strategy in modo conciso.

4.2.1 Classi annidate, interne, locali, anonime

Definition 4.2.1: Classi in scope interni

Sia Java sia C++ permettono di definire una classe dentro un'altra. In C++ è solo una questione di *visibilità e naming*. In Java la distinzione è più profonda:

- **nested (static)**: dentro un'altra classe solo per organizzazione/naming; può usare solo i membri *static* dell'esterna. Si dichiara con `static`.
- **inner (non static)**: ogni istanza è legata a un'*istanza* della classe che la contiene e può accedere ai membri d'istanza di quest'ultima. Non può avere membri static.
- **local**: dichiarata dentro un *blocco*/metodo; visibile solo lì; accede alle variabili locali `final`/effectively final.
- **anonymous**: classe locale *senza nome*, dichiarata e istanziata in un'unica espressione; una sola istanza. Usata quando serve una classe una volta sola.

```
public class X {
    int instanceVar;
    static int staticVar;

    private class Inner {                // INNER: legata all'istanza di X
        int y;
        Inner() { y = instanceVar + staticVar; }   // accede a entrambi
    }
    private static class Nested {        // NESTED: solo membri static
        int x;
        Nested() { x = staticVar; /* instanceVar -> ERRORE */ }
    }
}
```

In C++ esiste solo la versione "nested" (naming/visibilità); non c'è il legame automatico con l'istanza dell'esterna:

```
class X {
public:
    int instanceVar;
    static int staticVar;
    class Nested {
        int x;
        Nested() : x(staticVar) { /* instanceVar++ -> ERRORE */ }
    };
};
```

Classi annidate in Python

Python permette di definire una classe sia dentro un'altra classe sia dentro una funzione, ma la *semantica* è diversa da Java. La regola da ricordare:

Proposition 4.2.1 La classe annidata in Python ≈ nested *static* di Java

Una classe definita dentro un'altra è semplicemente un **attributo** della classe esterna (`Outer.Inner`). **Non** riceve alcun riferimento automatico all'istanza esterna: si comporta quindi come una *static nested class* di Java, mai come una *inner class*. La parola chiave `static` non esiste (e non serve) per le classi.

Note:

Gotcha fondamentale (scope di classe non annidati). Il corpo di una classe annidata *non vede* i nomi definiti nel corpo della classe esterna: in Python solo gli scope di *funzione* (via closure), globale e built-in fanno catena (LEGB), non quelli di classe. Verificato: `y = x` dentro `Inner` solleva `NameError`.

```python
class Outer:
    x = 1
    class Inner:
        # y = x        # -> NameError: lo scope della classe esterna NON e' visibile
        pass

o = Outer.Inner()      # Inner e' solo un attributo di Outer
# o non ha alcun riferimento automatico a un'istanza di Outer

# "inner class" con accesso all'esterna: si passa self ESPLICITAMENTE
class Container:
    def __init__(self): self.value = 42
    class Handle:
        def __init__(self, outer): self.outer = outer
        def show(self): return self.outer.value
    def make(self): return Container.Handle(self)   # legame esplicito
```

Classe locale (dentro una funzione). A differenza degli scope di classe, gli scope di *funzione* in Python si annidano: una classe definita dentro una funzione può leggere le variabili locali della funzione, sia nel corpo della classe sia nei metodi (**closure**), senza bisogno di `final` come in Java.

```python
def make_adder(n):
    class Adder:            # classe LOCALE
        base = n            # il corpo classe legge il locale n della funzione
        def add(self, x): return x + n   # metodo: closure su n
    return Adder()

make_adder(10).add(5)       # 15
```

Classi anonime: non esistono (e non servono). Python non ha una sintassi per le classi anonime. Al loro posto:

- le `lambda` danno *funzioni* anonime (una sola espressione);
- `type("Nome", (Base,), {...})` crea una classe al volo (ma vuole comunque un nome);
- nella stragrande maggioranza dei casi non serve nulla di tutto ciò: dove Java usa una classe anonima che implementa un'interfaccia a metodo singolo (es. `ActionListener`), Python passa direttamente una funzione (funzioni first-class + duck typing).

Note:

Perché in Python le nested class sono rare. Il meccanismo idiomatico di *namespacing* in Python è il **modulo** (un file `.py`), non la classe annidata. Le classi annidate si usano quasi solo per legare strettamente un tipo di supporto alla sua classe (es. `Model.Meta` in Django) — proprio l'uso "static nested" di Java.

Classi annidate in C

In C non esistono classi: l'analogo è la definizione di una `struct` dentro un'altra `struct` (o dentro una funzione). Ma le regole di scope sono molto diverse da C++/Java.

Proposition 4.2.2 In C il tag della struct annidata "leaka" nello scope esterno

Contrariamente a C++ (dove `Inner` è `Outer::Inner`, scoped nella classe), in C il *tag* di una struct annidata viene "issato" nello scope del file: diventa un tipo globale visibile ovunque. Non c'è quindi alcun vero *namespacing*. (Verificato: il codice sotto compila in C, mentre in C++ `struct Inner leaked;` dà "incomplete type".)

```c
struct Outer {
    int x;
    struct Inner { int y; } inner;   /* struct annidata */
```

```
};

struct Inner leaked;     /* C: OK, il tag Inner e' visibile a livello di file */
                         /* C++: ERRORE, servirebbe Outer::Inner */
```

- **Struct locale**: una `struct` definita dentro una funzione è invece a scope di blocco (visibile solo lì), l'unico caso in cui C offre un vero incapsulamento del tipo.
- **Nessuna inner class**: per dare a una struct "annidata" accesso ai dati di quella esterna si passa un *puntatore* all'esterna esplicitamente (come per lo Strategy e le funzioni virtuali in C).
- **Nessuna classe anonima**: attenzione, il C11 ha le *anonymous struct/union*, ma sono *membri* aggregati senza nome (servono per il layout, es. accedere a `s.a` invece di `s.u.a`), non hanno nulla a che vedere con le classi anonime di Java.

Quadro comparativo

Concetto (Java)	Java	C++	Python	C
Nested (static)	`static class`	`class` annidata (scoped)	classe annidata (default)	struct annidata (tag *leaka*)
Inner (legata all'istanza)	`class` (ref. esterna implicita)	non esiste (ref. esplicito)	non esiste (passi `self`)	non esiste (passi puntatore)
Local (in funzione)	sì (inner, locali `final`)	sì	sì (closure sui locali)	sì (struct block-scoped)
Anonima	sì (`new I(){...}`)	lambda (C++11)	no (lambda / `type()`)	no (solo anon. struct *member*)
Namespacing preferito	package + nested	namespace + nested	**modulo**	file + prefissi nomi

4.2.2 Interfacce

Definition 4.2.2: Interfaccia (Java)

Un tipo riferimento che dichiara *signature* di metodi (astratti), costanti, e — da Java 8 — metodi `default` e `static`. Non ha variabili d'istanza e non può essere istanziata: le classi la `implements` fornendo il corpo dei metodi astratti.

Proposition 4.2.3 Perché le interfacce (e non l'ereditarietà multipla)

Le interfacce permettono a classi *non* imparentate di condividere un contratto comune **senza** una superclasse astratta condivisa. Java ammette `implements` di *più* interfacce (ereditarietà multipla di *tipo*) evitando le ambiguità dell'ereditarietà multipla di *stato* tipica del C++ (il "diamond problem").

Definition 4.2.3: Default method

Metodo di interfaccia con un corpo (`default`). Serve a **far evolvere un'interfaccia** senza rompere le classi che già la implementano: aggiungere un metodo astratto le romperebbe, un `default` no (fornisce un'implementazione di fallback ereditabile).

```
public interface DoIt {
    void doSomething(int i, double x);          // astratto
    int  doSomethingElse(String s);             // astratto
    default boolean didItWork(int i, double x, String s) {  // aggiunto dopo
        return true;                            // NON rompe le vecchie classi
    }
}
```

Proposition 4.2.4 Risoluzione dei conflitti tra default method

Se una classe eredita più default con la stessa signature, Java applica:

1. i metodi *d'istanza* di una superclasse battono i default d'interfaccia ("class wins");
2. i metodi già sovrascritti da altri candidati vengono ignorati;
3. se resta un conflitto irrisolubile, **errore di compilazione**: la classe *deve* fare override esplicito, potendo invocare `Interfaccia.super.metodo()`.

4.2.3 Classi anonime e listener

Il caso d'uso principe è la gestione degli eventi (GUI): un `JButton` ha bisogno di un oggetto che implementi `ActionListener` (interfaccia con il solo metodo `actionPerformed`). Le slide mostrano tre modi di fornire la strategia di reazione al click:

```
// 1) la classe contenitore implementa direttamente l'interfaccia
public class SimpleCalc implements ActionListener {
    public void actionPerformed(ActionEvent e) { /* ... */ }
}

// 2) inner class dedicata (accede ai campi dell'esterna, parametrizzabile)
private class OperatorAction implements ActionListener {
    private int operator;
    OperatorAction(int op) { this.operator = op; }
    public void actionPerformed(ActionEvent e) { /* usa operator */ }
}

// 3) classe anonima: la strategia definita sul posto, dove serve
equalsButton.addActionListener(new ActionListener() {
    @Override public void actionPerformed(ActionEvent e) { /* ... */ }
});
```

Note:

La classe anonima *sembra* passare "un pezzo di codice" come parametro, ma in realtà crea una classe a tutti gli effetti, ne istanzia un oggetto e passa *quell'oggetto*. È il ponte concettuale verso le **lambda** (Java 8), che rendono lo Strategy ancora più conciso quando l'interfaccia ha un solo metodo (*functional interface*): `equalsButton.addActionListener(e -> { ... });`

4.3 Lo Strategy nei quattro linguaggi

Qui sta il cuore del confronto richiesto dal prof: *lo stesso pattern*, quattro realizzazioni molto diverse a seconda di cosa il linguaggio offre.

4.3.1 Java — interfaccia + (anonima/lambda)

```
interface Strategy { int execute(int a, int b); }

class Context {
    private Strategy s;
    Context(Strategy s) { this.s = s; }
    int run(int a, int b) { return s.execute(a, b); }   // late binding
}

Context add = new Context((a, b) -> a + b);   // lambda = strategia
Context mul = new Context((a, b) -> a * b);
```

4.3.2 C++ — classe astratta, `std::function` o template

C++ offre più alternative: la classica (classe base astratta + virtual), oppure funzioni di prima classe con `std::function`, oppure la *policy-based design* con i template (strategia scelta a compile-time, zero-overhead).

```cpp
// (a) classica: interfaccia = classe astratta con metodo virtuale puro
struct Strategy { virtual int execute(int, int) = 0; virtual ~Strategy() {} };
struct Add : Strategy { int execute(int a, int b) override { return a + b; } };

// (b) moderna: la strategia e' una funzione (first-class)
#include <functional>
std::function<int(int,int)> s = [](int a, int b){ return a + b; };

// (c) policy-based: strategia come parametro di template (compile-time)
template <class Policy>
struct Context { int run(int a, int b) { return Policy::execute(a, b); } };
```

Definition 4.3.1: Funzione virtuale pura / classe astratta (C++)

Un metodo dichiarato `= 0` è *puro*: la classe che lo contiene è **astratta** (non istanziabile) e funge da interfaccia. È l'equivalente C++ dell'`interface` di Java; combinando più classi astratte si ottiene l'ereditarietà multipla di tipo.

4.3.3 C — puntatori a funzione (lo Strategy "originale")

In C la strategia è semplicemente un **puntatore a funzione** passato al codice cliente: è la forma più antica ed essenziale del pattern.

```c
typedef int (*Strategy)(int, int);       // tipo "strategia"

int run(Strategy s, int a, int b) { return s(a, b); }

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main() {
    printf("%d\n", run(add, 3, 4));      // 7
    printf("%d\n", run(mul, 3, 4));      // 12
}
```

Note:

Il prof osserva che in Java lo Strategy passa *per forza* da interfacce e overriding "proprio perché non usiamo puntatori a funzione". In C accade l'opposto: non essendoci oggetti né override, il puntatore a funzione è *l'unico* strumento — ed è sufficiente.

4.3.4 Python — funzioni di prima classe (quasi banale)

In Python le funzioni sono oggetti di prima classe: lo Strategy si riduce a passare una funzione (o qualunque *callable*). Non servono né interfacce né classi.

```python
def run(strategy, a, b):
    return strategy(a, b)

run(lambda a, b: a + b, 3, 4)    # 7
run(lambda a, b: a * b, 3, 4)    # 12
```

Proposition 4.3.1 Perché lo stesso pattern cambia forma

- Dove ci sono **funzioni di prima classe** (C, Python, C++ moderno) lo Strategy è "passa una

funzione": leggero, poche righe.

- Dove il linguaggio è **puramente a oggetti senza puntatori a funzione** (Java classico) serve incapsulare la strategia in un *oggetto* che implementa un'interfaccia — da cui l'importanza di classi anonime e lambda per non appesantire il codice.

Il pattern è lo stesso; cambiano gli *ingredienti* che il linguaggio mette a disposizione.

4.4 Ereditarietà e polimorfismo: quadro comparativo

Lo Strategy poggia sul polimorfismo, quindi conviene chiudere sistematizzando come i quattro linguaggi gestiscono ereditarietà e interfacce.

Aspetto	**C**	**C++**	**Java**	**Python**
Ereditarietà	assente	multipla (di stato)	singola (di stato)	multipla (di stato)
Interfacce	assenti	classi astratte (virtual puro)	`interface` (multiple)	ABC / duck typing
Diamond problem	—	sì → risolto con `virtual` inheritance	evitato: stato singolo, tipo multiplo	risolto dalla MRO (linearizzazione C3)
Metodo astratto	—	`virtual f() = 0;`	`abstract` / metodo d'interfaccia	`@abstractmethod`
Strategia idiomatica	puntatore a funzione	virtual / `std::function` / template	interfaccia + lambda	funzione / callable

Definition 4.4.1: Diamond problem ed ereditarietà virtuale

Se `D` eredita da `B` e `C`, entrambe derivate da `A`, in C++ `D` contiene *due* copie di `A` (ambiguità sui membri di `A`). Si risolve con l'**ereditarietà virtuale** (`class B : virtual public A`), che fa condividere un'unica `A`. Java evita il problema alla radice: erediti lo *stato* da una sola classe, e più interfacce (che stato non hanno). Python usa la **MRO C3** per linearizzare l'ordine di ricerca dei metodi.

Chapter 5

Tabelle riassuntive per l'esame

Riepilogo "a colpo d'occhio" dei confronti chiesti dal professore. Queste tabelle raccolgono, per ciascun argomento delle slide, la posizione dei quattro linguaggi.

5.1 Modello di esecuzione e tipizzazione

	C	**C++**	**Java**	**Python**
Compilazione	nativo	nativo	bytecode + JVM	bytecode + VM
Tipizzazione	statica debole	statica forte	statica forte	dinamica forte
Type-check tipi	compile-time	compile-time	compile-time	run-time
Memoria	manuale	manuale + RAII	GC	GC (refcount)
Velocità tipica	massima	massima	alta (JIT)	bassa

5.2 Polimorfismo e dispatch (funzioni virtuali)

	C	**C++**	**Java**	**Python**
Dispatch di default	early (statico)	early (statico)	late (dinamico)	late (dinamico)
Metodo virtuale	simulato (ptr a funz.)	`virtual` esplicito	sempre	sempre
Bloccare l'override	—	non-`virtual` / `final` (C++11)	`final`	convenzione / `__`
Meccanismo	tabella manuale	vtable + vptr	tabella metodi JVM	lookup MRO / dict
Overhead	nullo	solo se virtual	sempre presente	interpretazione

5.3 Genericità (template / generics)

	C	**C++**	**Java**	**Python**
Meccanismo	macro / `void*`	template	generics	duck typing
Quando	preproc. / runtime	compile-time	compile-time	runtime
Strategia	sostituzione testo	monomorfizzazione	type erasure	nessuna
Type-safe	no	sì (all'uso)	sì (poi erasa)	no (solo hint)
Primitivi	sì	sì	no (boxing)	n/a
Info a runtime	—	sì	no	sì (dinamico)

5.4 OOP: ereditarietà e interfacce (strategy)

	C	**C++**	**Java**	**Python**
OOP nativo	no	sì	sì	sì
Ereditarietà multipla	—	sì (stato)	solo interfacce	sì (MRO)
Interfaccia	—	classe astratta	`interface`	ABC / protocol
Strategy idiomatico	ptr a funzione	virtual / functional / template	interfaccia + lambda	callable
Classi anonime	—	lambda (C++11)	sì	lambda

5.5 Le domande "trappola" tipiche del prof

Proposition 5.5.1 Cose da saper rispondere a voce

- *Perché C++ ha* `virtual` *e Java no?* ⇒ efficienza / zero-overhead: C++ non fa pagare il late binding a chi non lo usa; Java preferisce l'uniformità.
- *Differenza tra template C++ e generics Java?* ⇒ monomorfizzazione (codice per ogni tipo, runtime type noto) vs type erasure (un solo bytecode, tipo cancellato).
- *Come si fa polimorfismo/strategy in C?* ⇒ puntatori a funzione (dentro struct = vtable manuale).
- *Perché Python non ha bisogno di generics?* ⇒ duck typing: i tipi si controllano a runtime, una funzione lavora su qualunque oggetto "compatibile".
- *Overriding vs overloading vs hiding?* ⇒ stessa signature+virtuale (runtime) / stesso nome, signature diversa (compile-time) / (solo C++) omonimo nella derivata che nasconde la base.
- *A cosa serve la vtable?* ⇒ a realizzare il late binding: ogni oggetto ha un vptr alla tabella dei puntatori ai metodi virtuali della sua classe reale.
- *Perché i default method nelle interfacce Java?* ⇒ per far evolvere un'interfaccia senza rompere le classi che già la implementano.

Fine degli appunti trasversali sui linguaggi.
Argomenti coperti: funzioni virtuali & late binding, template & generics, strategy pattern — confrontati su C, C++, Java, Python.